서울시립대학교 · 전자전기컴퓨터설계실험II

LAB5 실험 전 보고서

FPGA5 / MCU1 — 피아노 · 신호등 제어 · ADC · STM32 인터럽트와 PWM (제공 예제 3개 · 과제3 · 과제4)

이름
형종희

학번
2023440145

조
C조

작성일
2026-10-17

실험일
2026-10-19

FPGA 보드
Combo II-DLD S75 · xc7s75fgga484-1

MCU 보드
NUCLEO-G474RE

도구
Vivado 2026.1 · Icarus Verilog 12.0 · STM32Cube (GCC 14.3.1)

📦 GitHub 제출 정보

항목	값
저장소	https://github.com/hyeongjonghee/electronics2-LABs (Public)
LAB5 설계 소스 커밋	`569be96` — FPGA 3개 회로의 RTL · 테스트벤치 · Icarus/XSim 파형(VCD) · 합성 보고서, STM32 과제3·4 코드 (31개 파일)
LAB5 경로	`LAB5/circuits/LAB5-01~03/` , `LAB5/stm32/task3~4/` , `LAB5/LAB5_실험전보고서.html` · `.pdf`
실험 전 보고서 태그	`lab5-prelab` — 이 보고서(HTML · PDF)가 들어간 커밋을 가리킴. 최종 제출 태그 `lab5-submit` 은 실험 후 보고서 커밋에 따로 건다.
STM32 기반 프로젝트	`Glaysia/ece2_474` 태그 `v1.1.0` = 커밋 `dd94f01` (수업 지정 버전). 학생이 바꾼 파일은 `Core/Src/ece2_474_routine.c` 하나이며, 과제별 사본을 위 저장소에 보관.

0. 사전 결과 요약

LAB5는 10/19 수업(FPGA5 / MCU1)의 실습이다. 수업 매뉴얼 STM32_1에 따라 **FPGA 개별 실습 3개**(피아노 → 신호등 → ADC)와 **STM32 제공 예제 3개, 직접 작성 과제 2개**(과제3 · 과제4)를 준비했다. FPGA 회로는 Icarus와 Vivado XSim 두 시뮬레이터에서 수업 테스트벤치의 자기검사를 모두 통과했고, 합성까지 마쳤다. STM32 과제는 실제 프로젝트로 빌드했고, 보드 없이 동작을 확인하려고 **저장소의 C 파일을 그대로 PC에서 컴파일해 가짜 HAL 함수(시계 · LED · 타이머)와 함께 실행**하는 모의 실행으로 타이밍을 검증했다.

#	실험	검증	결과	자원 / 빌드
1	피아노 (FPGA)	Icarus · XSim 자기검사	PASS 8음 반주기 · 무음 · 다중 키 · 리셋	LUT 646 · FF 43 · IOB 11
2	신호등 제어 (FPGA)	Icarus · XSim 자기검사	PASS 4방향 × 13단계 · 보행자 점멸 · 점멸 모드	LUT 33 · FF 32 · IOB 27
3	ADC (FPGA)	Icarus · XSim 자기검사	PASS 직렬 프레임 3개 · 명령 비트 · 12비트 값	LUT 60 · FF 72 · IOB 30
4	STM32_1 제공 예제 3개	호스트 모의 실행	PASS 버튼 토글 · 1초 점멸 · PWM 50 %	실험 당일 실행
5	STM32_1 과제3 · ISR로 간격 변경	빌드 + 호스트 모의 실행	PASS 1→2→4→8→1 s, 떨림 무시	FLASH 25,976 B (4.95 %) · RAM 2,656 B
6	STM32_1 과제4 · 버튼으로 PWM 비율	빌드 + 호스트 모의 실행	PASS 25→50→75→25 %	FLASH 26,308 B (5.02 %) · RAM 2,648 B

공통 사항

- **설계 소스 출처**: FPGA 설계 모듈과 테스트벤치는 수업 저장소 `docs/FPGA_additional.md` (v1.1.0)를 그대로 사용했다. 테스트벤치는 고치지 않았고, 파형 저장은 Icarus에서는 별도 파일 `sim/dump.sv`, XSim에서는 Tcl 명령 `log_vcd`로 했다.
- **두 시뮬레이터의 결과 비교**: 같은 테스트벤치를 Icarus와 XSim으로 돌려 저장한 VCD를 비교했다. 설계 출력의 모든 변화 시각과 값이 같았다. 다른 점은 **시간 0에 Icarus가 조합 출력을 X로 두는 것**뿐이다. Icarus는 `always @*`를 입력이 처음 바뀔 때까지 실행하지 않기 때문이며, 첫 입력 변화 뒤에는 차이가 없다.
- **MCU 클록**: 수업 템플릿의 `SystemClock_Config()`는 HSI 16 MHz → PLL(÷2 · ×25 · ÷2) = **100 MHz**이고, APB1 분주가 1이므로 TIM3 · TIM6의 입력 클록도 100 MHz이다.

실험 당일 확인할 것 — ① **FPGA 핀**: 피아노의 키패드 배치와 리셋, 신호등 LED 24개(교재 3.5절 p.644), ADC 직렬선과 채널 선택(교재 3.4절 p.615)은 아직 교재 핀표와 대조하지 않았다. 당일 교재 사진으로 **신호 이름 단위로 한 줄씩** 대조한 뒤 XDC를 쓰고 구현 · 비트스트림을 만든다. 그래서 이 보고서의 ⑥은 합성까지만 있다. ② **PWM 측정**: PB4와 GND에 오실로스코프를 연결해 주기 1 ms와 HIGH 시간 0.25/0.50/0.75 ms를 확인한다.

목차

1. 실험 1 · 피아노
2. 실험 2 · 신호등 제어
3. 실험 3 · ADC
4. 실험 4 · STM32_1 제공 예제
5. 실험 5 · STM32_1 과제3
6. 실험 6 · STM32_1 과제4
7. 고찰
8. 참고문헌

1. 실험 1 · 피아노 (piano)

① 목적

키 8개 중 하나를 누르면 해당 음(도 262 Hz ~ 높은 도 523 Hz)의 사각파를 피에조로 내보내는 회로를 만든다. 50 MHz 클록을 세어 원하는 주파수를 만드는 **분주** 방법을 익히고, 아무 키도 누르지 않거나 두 개 이상 누르면 소리가 나지 않는지 확인한다.

② 이론 · 설계 계산

사각파를 클록으로 만드는 법. 주파수 f인 사각파는 한 주기 $1/f$ 동안 HIGH 반, LOW 반이다. 그러니 출력을 **반주기마다 한 번씩 뒤집으면** 된다. 반주기를 클록 칸 수로 바꾸면

$$N = \left\lfloor \frac{f_{clk}}{2f} \right\rfloor, \qquad f_{\text{실제}} = \frac{f_{clk}}{2N}$$

이다. `square_tone`은 `count`를 0부터 $N-1$까지 세고, $N-1$에 닿으면 0으로 돌아가며 `tone`을 뒤집는다. 예를 들어 라(440 Hz)는 $N = 50{,}000{,}000/880 = 56{,}818$칸이다. 손목시계 초침이 아니라 "칸을 몇 번 세면 뒤집는다"는 약속으로 소리를 만드는 셈이다.

정수로 잘라서 생기는 오차. N을 정수로 내리면 실제 주파수가 아주 조금 높아진다. 음 높이 차이는 cent(반음의 1/100)로 재며 $1200\log_2(f_{\text{실제}}/f)$이다. 아래 표처럼 50 MHz에서는 오차가 0.03 cent 이하로, 사람이 구분할 수 있는 수준(수 cent)보다 훨씬 작다.

키 처리 규칙 (`note_select`). `case(key)`는 8비트 전체를 비교하므로 정확히 한 비트만 1일 때만 음을 고른다. 0이거나 두 비트 이상이면 `default`로 가서 주파수 0(무음)이 된다. 또 `square_tone`은 주파수가 바뀌는 순간 `count`와 `tone`을 0으로 되돌려, 새 음이 항상 같은 위상(LOW)에서 시작한다.

시뮬레이션 축소. TB는 실행을 빠르게 하려고 `CLK_HZ=10000`으로 바꿔 넣는다. 이때 $N = \lfloor 10000/(2f) \rfloor$은 19 · 17 · 15 · 14 · 12 · 11 · 10 · 9칸이고, TB 클록이 10 ns이므로 반주기는 190 ~ 90 ns가 된다.

key (hex)	음	f (Hz)	반주기 N (50 MHz)	실제 f (Hz)	오차 (cent)	N (TB 10 kHz)	VCD 측정 반주기	판정
80	도 C4	262	95,419	262.00	+0.015	19	190 · 190 · 190 ns	일치
40	레 D4	294	85,034	294.00	+0.000	17	170 · 170 · 170 ns	일치
20	미 E4	330	75,757	330.00	+0.013	15	150 · 150 · 150 ns	일치
10	파 F4	349	71,633	349.00	+0.006	14	140 · 140 · 140 ns	일치
08	솔 G4	392	63,775	392.00	+0.014	12	120 · 120 · 120 ns	일치
04	라 A4	440	56,818	440.00	+0.006	11	110 · 110 · 110 ns	일치
02	시 B4	494	50,607	494.00	+0.010	10	100 · 100 · 100 ns	일치
01	높은 도 C5	523	47,801	523.00	+0.005	9	90 · 90 · 90 ns	일치

VCD 측정 반주기: XSim VCD에서 각 키 구간의 piezo 반전 시각 간격을 계산한 값. Icarus VCD도 같은 값이다.

③ 포트 · 보드 매핑

포트	방향	보드 장치	FPGA 핀	상태
clk_50mhz	입력	메인 클록 (다이얼 F = 50 MHz)	B6	**LAB3·4에서 검증**
piezo	출력	피에조 (PIEZO ENABLE 스위치 ON)	Y21	**매뉴얼 확인**
key[7:0] (도~높은 도)	입력	키패드 (누르면 HIGH) — 교재 배치 확인	후보: 키 1~8 = K4 · N8 · N4 · N1 · P6 · N6 · L5 · J2	**당일 교재 대조**
rst_p	입력	교재 피아노 예제의 리셋 입력	—	**당일 교재 대조**

키패드는 레벨 입력으로 그대로 쓴다. 키를 누르는 동안만 음이 나야 하므로 한 번 누름 펄스로 바꾸지 않는다. 접점이 떨리는 수 ms 동안은 주파수가 0과 해당 음 사이를 오가며 위상이 다시 시작되지만, 귀로 들리는 차이는 거의 없다.

④ RTL

piano.v (최상위)

```
`timescale 1ns/1ps
module piano #(parameter CLK_HZ=50_000_000)(
    input wire clk_50mhz, rst_p,
    input wire [7:0] key,
    output wire piezo
);
    wire [31:0] frequency_hz;
    note_select u_note(key, frequency_hz);
    square_tone #(.CLK_HZ(CLK_HZ)) u_tone(clk_50mhz,rst_p,frequency_hz,piezo);
endmodule
```

note_select.v

```
`timescale 1ns/1ps
// 버튼 입력에 대응하는 음의 주파수를 선택합니다.
// 보드의 버튼 입력은 안정된 HIGH 활성 신호로 전달합니다.
module note_select(input wire [7:0] key, output reg [31:0] frequency_hz);
```

```
    always @* begin
        case(key)
            8'h80: frequency_hz=262;
            8'h40: frequency_hz=294;
            8'h20: frequency_hz=330;
            8'h10: frequency_hz=349;
            8'h08: frequency_hz=392;
            8'h04: frequency_hz=440;
            8'h02: frequency_hz=494;
            8'h01: frequency_hz=523;
            default: frequency_hz=0; // 무입력이나 여러 버튼 입력은 무음
        endcase
    end
endmodule
```

square_tone.v

```
`timescale 1ns/1ps
module square_tone #(parameter CLK_HZ=50_000_000)(
    input wire clk_50mhz, rst_p,
    input wire [31:0] frequency_hz,
    output reg tone
);
    reg [31:0] count, previous_hz;
    wire [31:0] half_cycles = frequency_hz == 0 ? 1 :
                              CLK_HZ / (2 * frequency_hz);
    always @(posedge clk_50mhz) begin
        if(rst_p) begin count<=0; tone<=0; previous_hz<=0; end
        else if(frequency_hz==0 || frequency_hz!=previous_hz) begin
            count<=0; tone<=0; previous_hz<=frequency_hz;
        end
        else if(count >= half_cycles-1) begin count<=0; tone<=~tone; end
        else count<=count+1;
    end
endmodule
```

⑤ 시뮬레이션

수업 TB(`sim/tb.sv`)는 8개 음에 대해 ① `note_select` 의 주파수 표, ② 키를 바꾼 직후 piezo가 0인지, ③ 반주기의 4배 동안 매 클록 piezo가 예상값과 같은지를 검사한다. 이어서 두 키 동시 입력(`8'hC0`) → 무음, 키 뗌 → 40클록 무음, 리셋을 확인한다. 두 시뮬레이터 모두 `PASS piano: all 8 tones, silence, multiple keys, reset` 을 출력했다(종료 4,796 ns).

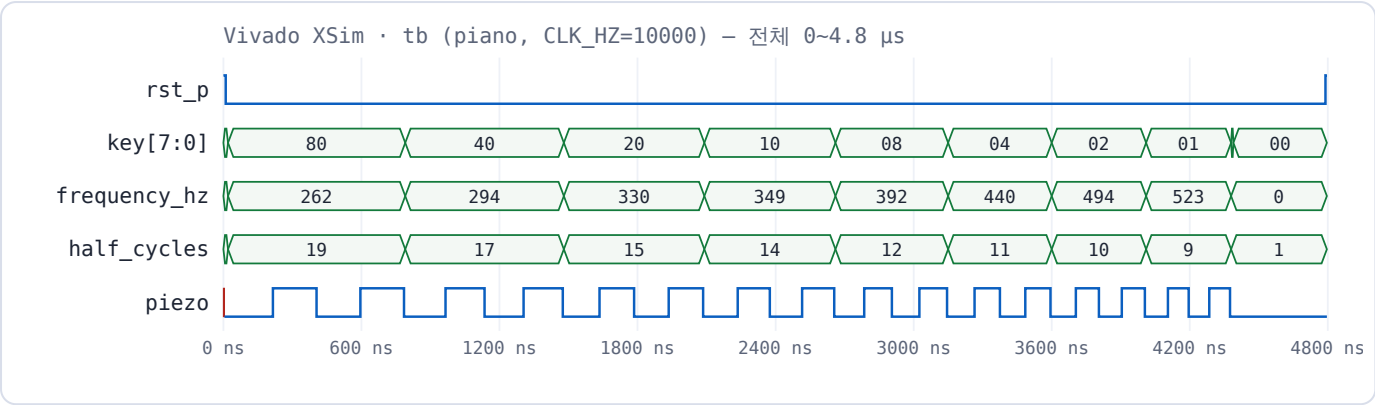


그림 1-1. Vivado XSim 파형(전체). 키가 도(80)→높은 도(01)로 바뀔 때마다 half_cycles가 19→9로 줄고 piezo 반전이 빨라진다. 끝부분 C0(두 키)과 00(무음)에서는 frequency_hz = 0, piezo = 0.

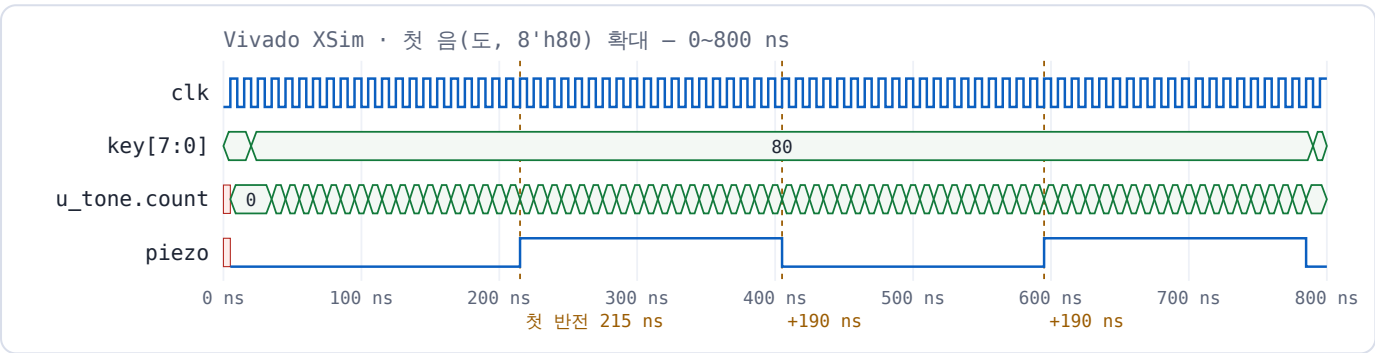


그림 1-2. XSim 첫 음 확대. 키가 바뀐 다음 클록에 count가 0으로 돌아가고, 19칸(190 ns)마다 piezo가 뒤집힌다. 첫 반전이 215 ns인 것은 키를 클록 하강 에지(20 ns)에 바꿔 첫 상승 에지(25 ns)에서 count를 0으로 맞춘 뒤 19칸을 세기 때문이다.

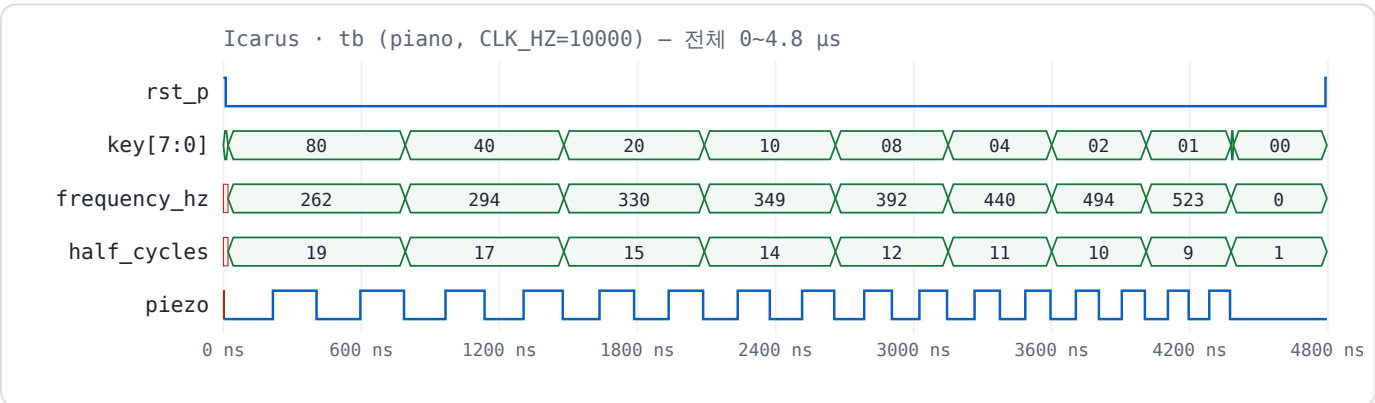


그림 1-3. Icarus 파형(전체). 시간 0의 빨간 X 칸(조합 출력 미정)을 빼면 그림 1-1과 같다.

⑥ 합성 결과 (핀 없이 합성까지)

LUT 646개 · FF 43개 · IOB 11개(클록 1 + 리셋 1 + 키 8 + 피에조 1) · DSP 0. 경고 0개.

LUT가 많은 이유. `CLK_HZ / (2*frequency_hz)`는 나누는 수가 입력(키)에 따라 바뀌는 **변수 나눗셈**이다. 합성기는 이것을 32비트 나눗셈 회로로 만들기 때문에, 같은 크기의 신호등(33개)보다 LUT가 20배 가까이 많다. 음이 8개뿐이므로 반주기 N을 미리 계산한 표(`case`로 상수 8개)를 쓰면 나눗셈기가 사라진다. 수업 코드는 그대로 두고, 실험 후 보고서에서 두 방식을 비교해 볼 예정이다.

⑦ 보드 실습 계획

1. 교재 피아노 예제 쪽에서 키패드 배치와 리셋 입력을 사진으로 확인하고 XDC를 쓴다(키 순서는 신호 이름 단위로 대조).
2. PIEZO ENABLE 스위치를 켜고 비트스트림을 올린다.
3. 키 8개를 차례로 눌러 음이 낮은 도부터 높은 도까지 올라가는지 듣는다. 두 키 동시 누름과 키를 뗀 상태에서 무음인지 확인한다.
4. 가능하면 피에조 핀을 오실로스코프로 재서 라(440 Hz)의 주기 2.273 ms를 확인한다.

2. 실험 2 · 신호등 제어 (traffic)

① 목적

네 방향 교차로의 차량 신호(빨강 · 노랑 · 초록 · 좌회전)와 보행자 신호(빨강 · 초록)를 1초 단위로 바꾸는 순서 회로를 만든다. 보행자 초록의 점멸과, 모든 신호를 노랑 · 보행자 빨강 점멸로 바꾸는 점멸 모드 (`flicker`)를 확인한다.

② 이론 · 설계 계산

새 클록 대신 tick. 1초마다 동작하는 회로를 만들 때 50 MHz를 나눠 1 Hz 클록을 새로 만들지 않는다. 대신 `second_tick` 이 50 MHz로 세다가 $CLK_HZ - 1$에 닿는 한 클록 동안만 `tick=1` 을 낸다. 나머지 회로는 모두 같은 50 MHz 클록을 쓰고 "tick일 때만 한 칸 진행"한다. 클록이 하나뿐이라 타이밍 분석이 단순해진다.

$$T_{tick} = \frac{CLK_HZ}{f_{clk}} = \frac{50{,}000{,}000}{50\,\mathrm{MHz}} = 1\,\mathrm{s}, \qquad blink = \left(count < CLK_HZ/2\right)$$

`blink` 는 1초 중 앞 0.5초만 1인 1 Hz, 듀티 50 % 신호다. 점멸이 필요한 LED는 이 신호와 AND한다.

순서. `traffic_sequence` 는 tick마다 `phase` 를 0→12로 올리고, 12 다음에는 0으로 돌아가며 `direction` 을 0→1→2→3→0으로 넘긴다. 한 방향은 13초, 한 바퀴는 $4 \times 13 = 52$초이다. `traffic_lights` 는 순수 조합 회로로, (방향, 단계, flicker, blink)에서 LED 24개를 계산한다.

phase (초)	현재 방향 차량	대응 보행자 초록	대응 보행자 빨강
0	초록	켜짐	꺼짐
1	초록	켜짐	꺼짐
2	초록	켜짐	꺼짐
3	초록	켜짐	꺼짐
4	초록	점멸 (blink)	꺼짐
5	초록	점멸 (blink)	꺼짐
6	초록	점멸 (blink)	꺼짐
7	초록	점멸 (blink)	꺼짐
8	초록	점멸 (blink)	꺼짐
9	초록	점멸 (blink)	꺼짐
10	초록	점멸 (blink)	꺼짐
11	노랑	꺼짐	빨강
12	노랑	꺼짐	빨강

방향별 마스크: 방향 0의 차량은 `4'b1000`, 보행자는 `4'b0100`이고, 방향이 하나 늘 때마다 한 칸씩 오른쪽으로 간다(방향 3의 보행자는 `1000`으로 돌아옴). 현재 방향이 아닌 차량은 모두 빨강이다. 좌회전 LED는 초록과 같은 값이다(`assign led_left = led_green`).

점멸 모드. `flicker=1`이면 차량 빨강 · 초록과 보행자 초록은 모두 끄고, 노랑 4개와 보행자 빨강 4개를 함께 blink로 점멸한다. 동시에 `traffic_sequence`는 방향 · 단계를 0으로 붙잡아, 점멸을 풀면 방향 0의 0초부터 다시 시작한다.

③ 포트 · 보드 매핑

포트	폭	보드 장치	FPGA 핀	상태
clk_50mhz	1	메인 클록	B6	검증됨
rst_p, flicker	1 · 1	교재 신호등 예제의 리셋 · 점멸 입력	—	당일 교재 대조
led_red · led_yellow · led_green · led_left	4 × 4	교재 3.5절(p.644) 신호등 LED (방향별)	—	당일 교재 대조
led_walk_red · led_walk_green	4 × 2	보행자 LED	—	당일 교재 대조

수업 문서에 따르면 출력 1은 "논리적으로 켜짐"이고, 장비 LED의 실제 극성은 핀 연결 단계에서 맞춘다. 교재 핀표에서 신호등 LED가 LOW에서 켜지는 구조이면 최상위에서 출력을 반전해 연결한다. 방향 번호(0~3)와 동 · 서 · 남 · 북의 대응도 교재 이름으로 맞춘다.

④ RTL

traffic.v (최상위)

```
`timescale 1ns/1ps
module traffic #(parameter CLK_HZ=50_000_000)(
    input wire clk_50mhz,rst_p,flicker,
    output wire [3:0] led_red,led_yellow,led_green,led_left,
    output wire [3:0] led_walk_red,led_walk_green
);
    wire tick,blink;
    wire [1:0] direction;
    wire [3:0] phase;
    second_tick #(.CLK_HZ(CLK_HZ)) u_tick(clk_50mhz,rst_p,tick,blink);
    traffic_sequence u_sequence(clk_50mhz,rst_p,tick,flicker,direction,phase);
    traffic_lights u_lights(direction,phase,flicker,blink,
        led_red,led_yellow,led_green,led_walk_red,led_walk_green);
    assign led_left=led_green;
endmodule
```

second_tick.v

```
`timescale 1ns/1ps
// 느린 동작은 새 클록 대신 한 클록 길이의 tick으로 만듭니다.
module second_tick #(parameter CLK_HZ=50_000_000)(
    input wire clk_50mhz, rst_p,
    output wire tick,
    output wire blink
);
    reg [31:0] count;
    assign tick=(count==CLK_HZ-1);
    assign blink=(count < CLK_HZ/2);
    always @(posedge clk_50mhz)
        if(rst_p || tick) count<=0;
        else count<=count+1;
endmodule
```

traffic_sequence.v

```
`timescale 1ns/1ps
// 각 방향에서 0~10초는 초록, 11~12초는 노랑입니다.
module traffic_sequence(
    input wire clk_50mhz,rst_p,tick,flicker,
    output reg [1:0] direction,
    output reg [3:0] phase
);
    always @(posedge clk_50mhz) begin
        if(rst_p || flicker) begin direction<=0; phase<=0; end
        else if(tick)
            if(phase==12) begin phase<=0; direction<=direction+1'b1; end
            else phase<=phase+1'b1;
    end
endmodule
```

traffic_lights.v

```
`timescale 1ns/1ps
module traffic_lights(
    input wire [1:0] direction,
    input wire [3:0] phase,
    input wire flicker,blink,
    output reg [3:0] red,yellow,green,walk_red,walk_green
);
    reg [3:0] vehicle_mask,pedestrian_mask;
    always @* begin
```

```
        // 방향별 차량과 보행자 LED를 선택합니다.
        case(direction)
            0: begin vehicle_mask=4'b1000; pedestrian_mask=4'b0100; end
            1: begin vehicle_mask=4'b0100; pedestrian_mask=4'b0010; end
            2: begin vehicle_mask=4'b0010; pedestrian_mask=4'b0001; end
            3: begin vehicle_mask=4'b0001; pedestrian_mask=4'b1000; end
        endcase
        red=4'b1111; yellow=0; green=0; walk_red=4'b1111; walk_green=0;
        if(flicker) begin
            red=0; yellow={4{blink}}; walk_red={4{blink}};
        end else begin
            if(phase<=10) begin green=vehicle_mask; red=~vehicle_mask; end
            else begin yellow=vehicle_mask; red=~vehicle_mask; end
            if(phase<=3) begin walk_green=pedestrian_mask; walk_red=~pedestrian_mask; end
            else if(phase<=10) begin
                walk_green=blink ? pedestrian_mask : 4'b0000;
                walk_red=~pedestrian_mask;
            end
        end
    end
endmodule
```

⑤ 시뮬레이션

TB는 `CLK_HZ=4`로 줄여 1초를 4클록(40 ns)으로 만든다. 4방향 × 13단계 × 4클록 = 208클록 동안 매 클록 LED 24개를 예상값과 비교하고(보행자 점멸은 각 초의 앞 2클록만 켜짐), 좌회전 = 초록을 확인한다. 이어서 점멸 모드 8클록, 리셋을 검사한다. 두 시뮬레이터 모두 `PASS traffic: full cycle, pedestrian blink, flicker, reset`을 출력했다(종료 2,186 ns).

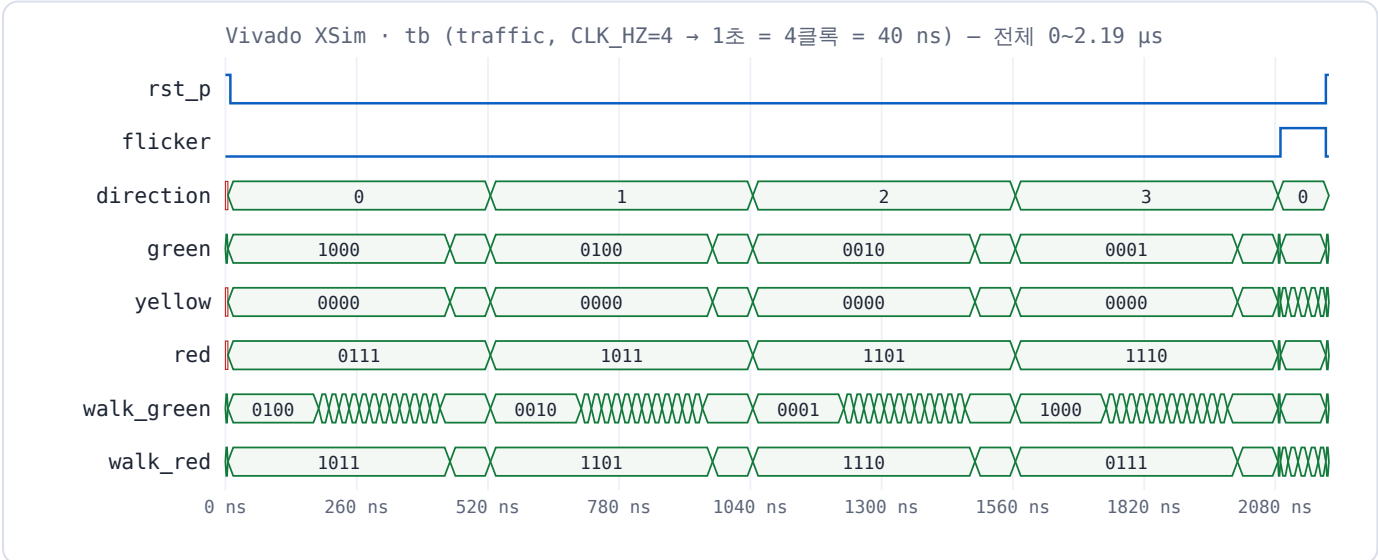


그림 2-1. Vivado XSim 파형(전체). direction 0→3 동안 green의 1이 한 칸씩 오른쪽으로 이동하고, 각 방향 끝 2초는 yellow로 바뀐다. 2.10 µs 근처의 flicker 구간에서는 yellow와 walk_red가 0000/1111로 함께 점멸한다.

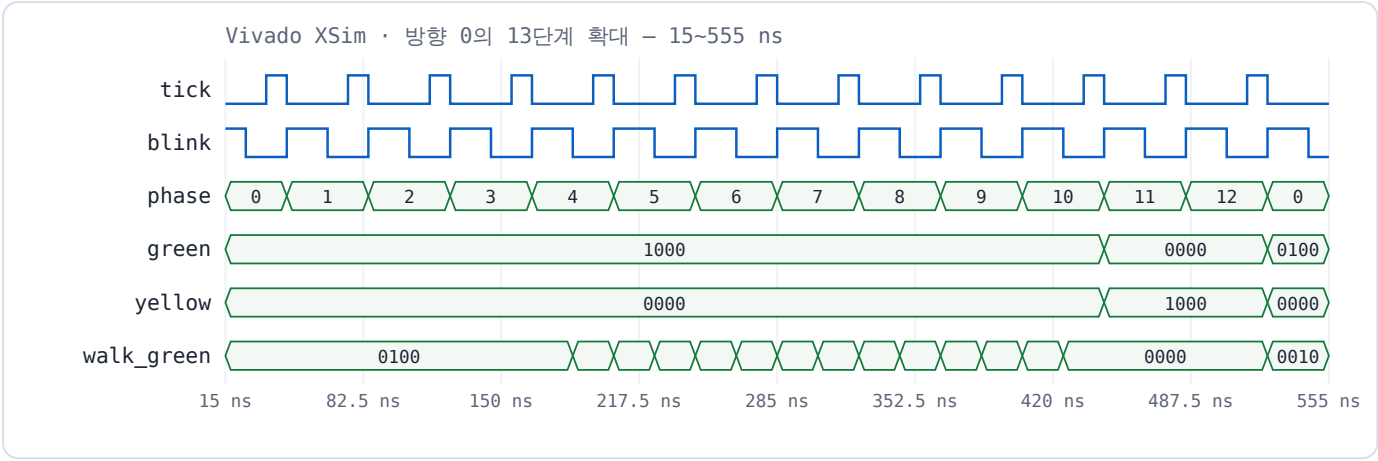


그림 2-2. XSim 방향 0 확대. tick마다 phase가 0→12로 오르고, phase 0~3은 보행자 초록 켜짐(0100), 4~10은 blink에 맞춰 0100/0000 점멸, 11~12는 차량 노랑(1000).

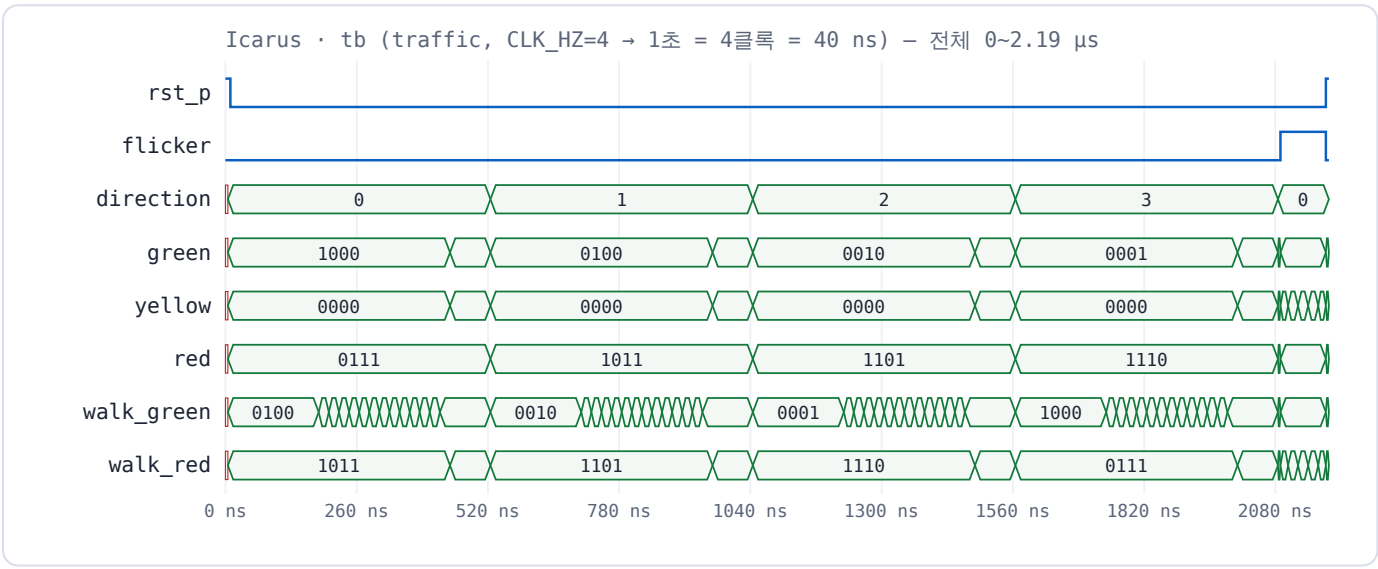


그림 2-3. Icarus 파형(전체). 시간 0의 X 칸을 빼면 그림 2-1과 같다.

⑥ 합성 결과 (핀 없이 합성까지)

LUT 33개 · FF 32개 · IOB 27개(클록 1 + 리셋 1 + flicker 1 + LED 24) · DSP 0. 경고 0개. FF는 1초 카운터와 방향 · 단계 레지스터이며, LED 출력은 조합회로라 FF를 쓰지 않는다.

⑦ 보드 실습 계획

1. 교재 p.644의 신호등 LED 핀과 극성, 리셋 · 점멸 입력을 사진으로 확인해 XDC를 쓴다.
2. 비트스트림을 올리고 스톱워치로 한 방향 13초(초록 11초 + 노랑 2초), 한 바퀴 52초를 잰다.
3. 보행자 초록이 4초 켜진 뒤 7초 점멸하는지, 좌회전이 초록과 함께 켜지는지 본다.
4. 점멸 입력을 켜서 노랑 · 보행자 빨강이 1 Hz로 함께 깜빡이는지, 끄면 방향 0부터 다시 시작하는지 확인한다.

3. 실험 3 · ADC (adc)

① 목적

FPGA가 외부 ADC 칩에 16비트 명령을 직렬로 보내고, 같은 시간에 돌아오는 16비트 응답에서 12비트 변환값을 꺼내 LED에 상위 8비트를 표시한다. 아날로그 입력을 바꾸면 LED 값이 바뀌는지 확인한다.
STM32 내부 ADC는 쓰지 않는다.

② 이론 · 설계 계산

직렬 통신(SPI 형식). 선 4개를 쓴다. `adc_ncs` (LOW = 칩 선택, 한 프레임 동안 LOW), `adc_sclk` (직렬 클록, 대기 시 HIGH), `adc_din` (FPGA → ADC 명령), `adc_dout` (ADC → FPGA 결과)이다. 클록이 내려갈 때 양쪽이 다음 비트를 내놓고, 올라갈 때 상대 비트를 읽는다. 두 사람이 줄을 하나씩 잡고 박자에 맞춰 동시에 쪽지를 한 장씩 주고받는 것과 같다. 16박자가 끝나면 FPGA는 명령 16비트를 다 보냈고 응답 16비트를 다 받았다.

명령 워드. `adc_command` 는 원본 예제의 비트 배치를 유지한다.

$$\text{command} = \{\,101,\ \text{ch}[2{:}0],\ 11,\ 0,\ 111,\ 0000\,\} = \texttt{16'hA370} \mid (\text{ch} \ll 10)$$

채널	계산한 명령	VCD에서 읽은 adc_din 16비트	VCD의 adc_dout 16비트	sample (하위 12비트)	led_out = sample[11:4]
2	`AB70`	`AB70`	`A5C3`	`5C3` = 1475	`5C`
5	`B770`	`B770`	`A5C3`	`5C3` = 1475	`5C`
7	`BF70`	`BF70`	`A5C3`	`5C3` = 1475	`5C`

VCD에서 읽은 값: XSim VCD에서 ncs가 LOW인 동안 sclk 상승 에지마다 din · dout을 샘플링해 이어 붙인 값(Icarus도 같음). 응답 0xA5C3은 TB의 ADC 모델이 매번 돌려주는 고정값이다.

속도. `serial_frame16` 은 50 MHz를 `HALF_CYCLES` 번 셀 때마다 sclk를 뒤집으므로 한 비트에 $2H$클록이 걸린다. VCD에서 프레임 시작 간격이 66클록($H = 2$)이었고, 이는 16비트 × 4클록 = 64클록에 다음 프레임 시작 준비 2클록을 더한 값이다. 따라서 보드 값 $H = 25$에서는

$$f_{sclk} = \frac{50\,\text{MHz}}{2H} = 1\,\text{MHz}$$

$$T_{frame} = (32H + 2)\,T_{clk} = 802 \times 20\,\text{ns} = 16.04\,\mu\text{s} \ \Rightarrow\ f_{sample} \approx 62.3\,\text{kS/s}$$

주의. 이 TB는 디지털 직렬 응답만 흉내 낸다. 실제 ADC는 전원 투입 뒤 초기화 프레임이 필요할 수 있고, 채널을 바꾼 명령의 결과가 다음 프레임에 나올 수 있다(수업 문서의 "채널 전환 시 결과 지연"). 칩 이름과 시간 조건은 당일 교재 3.4절과 장치 설명서로 확인한다.

③ 포트 · 보드 매핑

포트	방향 · 폭	보드 장치	FPGA 핀	상태
clk_50mhz	입력 1	메인 클록	B6	검증됨
led_out[7:0]	출력 8	LED1~8 (`led_out[7]` = LED1)	L4 · M4 · M2 · N7 · M7 · M3 · M1 · N5	LAB4에서 검증
adc_sclk · adc_ncs · adc_din · adc_dout	출력 3 · 입력 1	교재 3.4절(p.615) ADC 칩	—	당일 교재 대조
add_sel[2:0], rst_p	입력 3 · 1	교재 예제의 채널 선택 · 리셋 (DIP 스위치로 예상)	—	당일 교재 대조
sample[11:0], data_valid	출력 13	시뮬레이션 확인용	핀 배정 안 함	—

sample · data_valid는 보드에서 쓸 곳이 없다. XDC에 핀을 주지 않으면 구현 단계에서 "핀 미배정" 오류가 나므로, 당일에는 이 두 출력을 빼는 보드용 최상위를 따로 만들어 감싼다(LAB4의 `*_top.v` 와 같은 방식).

④ RTL

adc.v (최상위)

```
`timescale 1ns/1ps
module adc #(parameter HALF_CYCLES=25)(
    input wire clk_50mhz,rst_p,
    input wire [2:0] add_sel,
    output wire adc_sclk,adc_ncs,adc_din,
    input wire adc_dout,
    output reg [11:0] sample,
    output reg data_valid,
    output wire [7:0] led_out
);
    wire [15:0] command,rx_word;
    wire busy,done;
```

```
    reg start;
    adc_command u_command(add_sel,command);
    serial_frame16 #(.HALF_CYCLES(HALF_CYCLES)) u_serial(
        clk_50mhz,rst_p,start,command,adc_dout,
        adc_sclk,adc_ncs,adc_din,busy,done,rx_word);
    always @(posedge clk_50mhz) begin
        if(rst_p) begin start<=0; sample<=0; data_valid<=0; end
        else begin
            start<=!busy && !start;
            data_valid<=done;
            if(done) sample<=rx_word[11:0];
        end
    end
    assign led_out=sample[11:4];
endmodule
```

adc_command.v

```
`timescale 1ns/1ps
module adc_command(input wire [2:0] channel,output wire [15:0] command);
    // 원본 예제의 WR·SEQ·주소·전원·범위·코딩 비트 구성을 유지합니다.
    assign command={3'b101,channel,2'b11,1'b0,3'b111,4'b0000};
endmodule
```

serial_frame16.v

```
`timescale 1ns/1ps
// 16비트 직렬 송수신. 클록은 HIGH 대기, 하강 에지에서 준비, 상승 에지에서 수신합니다.
module serial_frame16 #(parameter HALF_CYCLES=25)(
    input wire clk_50mhz,rst_p,start,
    input wire [15:0] tx_word,
    input wire miso,
    output reg sclk,cs_n,mosi,busy,done,
    output reg [15:0] rx_word
);
    reg [31:0] count;
    reg [4:0] bit_index;
    reg [15:0] tx_latched;
    always @(posedge clk_50mhz) begin
        if(rst_p) begin
            sclk<=1; cs_n<=1; mosi<=0; busy<=0; done<=0;
            count<=0; bit_index<=15; tx_latched<=0; rx_word<=0;
        end else begin
            done<=0;
            if(!busy) begin
```

```
                if(start) begin
                    cs_n<=0; busy<=1; count<=0; bit_index<=15;
                    tx_latched<=tx_word; rx_word<=0; mosi<=tx_word[15];
                end
            end else if(count==HALF_CYCLES-1) begin
                count<=0;
                if(sclk) begin sclk<=0; mosi<=tx_latched[bit_index]; end
                else begin
                    sclk<=1;
                    rx_word<={rx_word[14:0],miso};
                    if(bit_index==0) begin busy<=0; cs_n<=1; done<=1; end
                    else bit_index<=bit_index-1'b1;
                end
            end else count<=count+1;
        end
    end
endmodule
```

⑤ 시뮬레이션

TB의 ADC 모델은 sclk 하강 에지마다 응답 0xA5C3을 한 비트씩 내놓고, 상승 에지마다 명령 비트를 받는다. `data_valid`가 올라갈 때마다 ① 받은 비트 수 16, ② 명령 = 0xA370 | (채널 ≪ 10), ③ sample · led_out을 검사한다. 채널을 2 → 5 → 7로 바꿔 프레임 3개를 확인한 뒤 리셋 상태를 검사한다. 두 시뮬레이터 모두 `PASS ADC: 3 serial frames, command bits, sample, reset`을 출력했다(종료 2,016 ns).

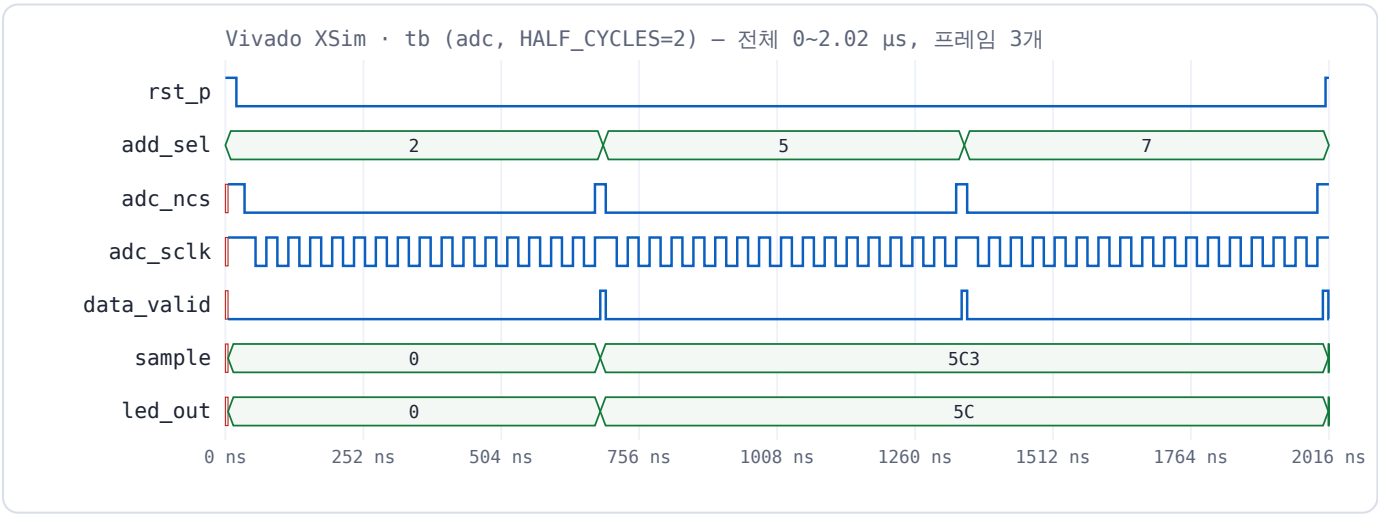


그림 3-1. Vivado XSim 파형(전체). ncs가 LOW인 구간마다 sclk 16펄스가 나가고, 프레임이 끝나면 data_valid가 한 클록 1이 되며 sample = 5C3, led_out = 5C로 바뀐다. 마지막 리셋에서 모두 0으로 돌아간다.

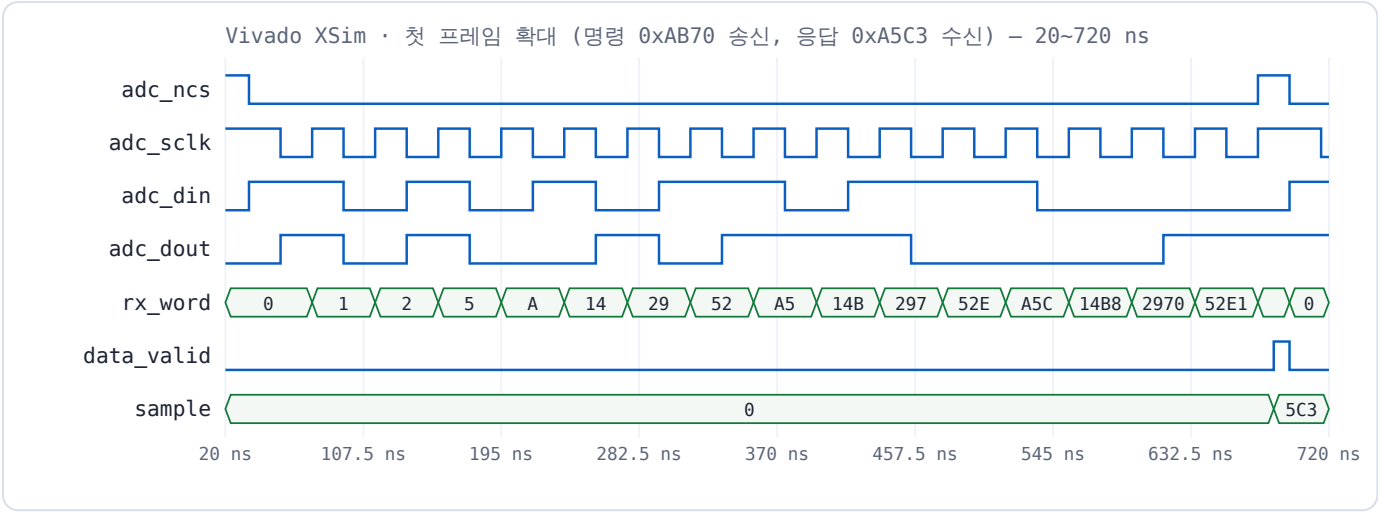


그림 3-2. XSim 첫 프레임 확대. sclk 하강 에지에서 adc_din(명령 1010 1011 0111 0000 = AB70)과 adc_dout(1010 0101 1100 0011 = A5C3)이 바뀌고, 상승 에지마다 rx_word가 한 비트씩 왼쪽으로 밀리며 채워진다.

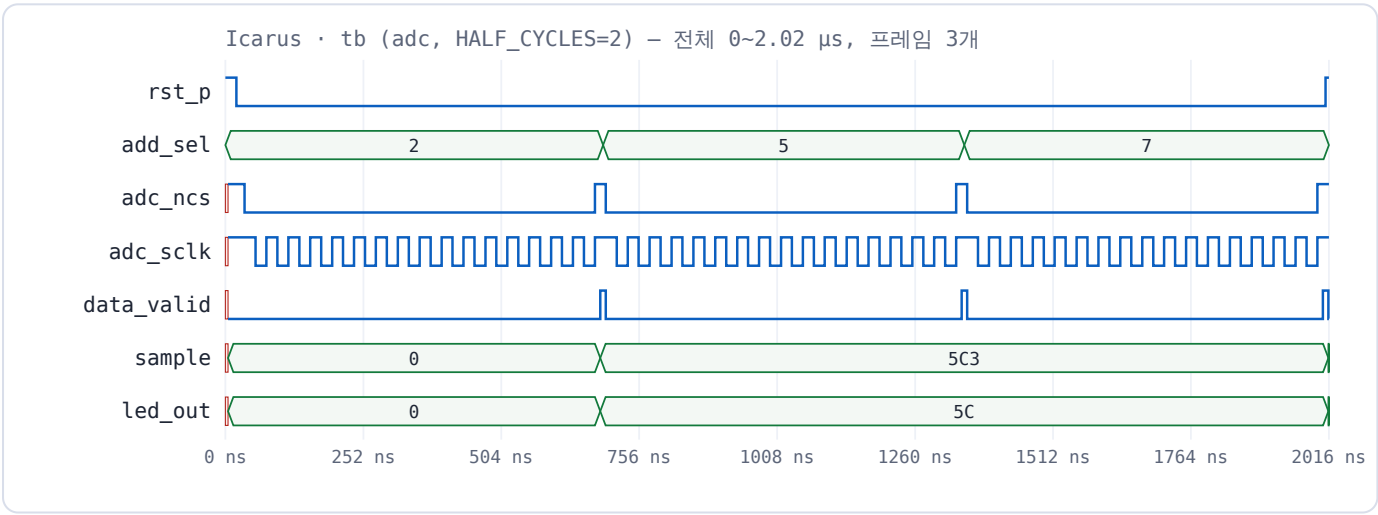


그림 3-3. Icarus 파형(전체). 그림 3-1과 모든 변화 시각이 같다.

⑥ 합성 결과 (핀 없이 합성까지)

LUT 60개 · FF 72개 · IOB 30개(클록 1 + 리셋 1 + 채널 3 + 직렬선 4 + sample 12 + valid 1 + LED 8) · DSP 0. 경고 0개. 보드용 최상위에서 sample · valid를 빼면 IOB는 17개가 된다.

⑦ 보드 실습 계획

1. 교재 p.615에서 ADC 칩, 직렬선 4개의 핀, 아날로그 입력(가변저항 등) 위치, 채널 선택 방법을 확인한다.
2. 보드용 최상위와 XDC를 쓰고 비트스트림을 만든다.
3. 아날로그 입력을 최소 → 최대로 돌리며 LED1~8이 0x00 → 0xFF 쪽으로 커지는지 본다.
4. 가능하면 ncs · sclk를 오실로스코프로 재서 sclk 1 MHz와 프레임 16 µs를 확인한다.

4. 실험 4 · STM32_1 제공 예제 3개

① 목적

과제 전에 매뉴얼의 제공 예제를 하나씩 따로 실행해, 메인 루프가 아니라 **인터럽트 서비스 루틴(ISR)**에서 일을 처리하는 구조와 타이머 · PWM 설정을 익힌다. 예제를 바꿀 때마다 이전 예제의 시작 호출과 동작 코드를 지운다(매뉴얼 "예제 시작과 전환").

② 이론 · 설계 계산

폴링과 인터럽트. STM32_0에서는 메인 루프가 계속 버튼을 읽었다(폴링). 인터럽트 방식에서는 버튼이 눌리는 순간(PC13 상승 에지, `GPIO_MODE_IT_RISING`) 하드웨어가 하던 일을 잠시 멈추고 `button_isr_p7()`을 부른다. 초인종이 울릴 때만 문으로 가는 것과 같아서, 메인 루프는 다른 일을 할 수 있다.

타이머 주기. 타이머 주파수는 입력 클록을 분주기(PSC+1)와 주기(ARR+1)로 나눈 값이다. 수업 템플릿의 설정값을 넣으면

$$f_{TIM6} = \frac{100\ \text{MHz}}{(9999+1)(999+1)} = 10\ \text{Hz}\ (100\ \text{ms})$$

$$f_{PWM} = \frac{100\ \text{MHz}}{(99+1)(999+1)} = 1\ \text{kHz}$$

$$D = \frac{CCR}{ARR+1} = \frac{500}{1000} = 50\ \%\quad (\text{HIGH}\ 0.5\ \text{ms}/1\ \text{ms})$$

우선순위. NVIC 우선순위는 숫자가 작을수록 높다. TIM6 = 4, UART3 = 5, UART2 = 6, 버튼(EXTI15_10) = 7, **SysTick = 15(가장 낮음)**이다. 그래서 ISR 안에서는 SysTick이 끼어들지 못해 `HAL_GetTick()` 값이 멈춰 있고, `HAL_Delay()`를 부르면 시간이 영영 지나지 않아 멈춘다. 매뉴얼이 ISR 안에서 HAL_Delay를 금지하는 이유다. 대신 `HAL_GetTick()`으로 현재 시각을 한 번 읽는 것은 괜찮다.

③ 포트 · 보드 매핑

예제	입력	출력	ece2_474_init()	동작 함수
① 버튼 ISR 토글	B1 파란 버튼 PC13	LD2 PA5	빈 상태	`button_isr_p7`
② TIM6 1초 점멸	TIM6 100 ms 인터럽트	LD2 PA5	`HAL_TIM_Base_Start_IT(&htim6)`	`tim6_isr_p4`
③ PWM 50 %	—	PB4 (TIM3 CH1) → 오실로스코프	PWM 시작 + 비교값 500	없음

④ 핵심 코드

예제 ① button_isr_p7 교체

```
static void button_isr_p7(void)
{
    static uint32_t last_ms = 0;
    uint32_t now_ms = HAL_GetTick();
    if (now_ms - last_ms >= 200U)
    {
        last_ms = now_ms;
        HAL_GPIO_TogglePin(LED_ece2_GPIO_Port, LED_ece2_Pin);
    }
}
```

예제 ② init에 TIM6 시작 + tim6_isr_p4 교체

```
HAL_TIM_Base_Start_IT(&htim6);      /* ece2_474_init() 안 */

static void tim6_isr_p4(void)
{
    static uint32_t count = 0;
    if (++count >= 10U)                  /* 100 ms x 10 = 1 s */
    {
        count = 0;
        HAL_GPIO_TogglePin(LED_ece2_GPIO_Port, LED_ece2_Pin);
    }
}
```

예제 ③ init에 PWM 시작

```
HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1);
__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, 500U);
```

⑤ 시뮬레이션 (호스트 모의 실행)

수업 템플릿에 예제 코드만 넣은 사본을 PC에서 컴파일해, 같은 가짜 HAL로 실행했다. 시나리오는 1.000 s에 버튼을 누르고 3 ms · 7 ms 뒤 접점 떨림 두 번, 2.000 s · 3.000 s에 다시 누름이다.

예제	LED 반전 시각 (ms)	해석
①	1000, 2000, 3000	버튼 3번 인정 · 떨림 2번(1003, 1007 ms) 무시
②	1000, 2000, 3000, 4000	버튼과 무관하게 1 s마다 반전 (TIM6 10회)
③	없음	PWM 시작 1회, 비교값 500 유지 → 50 %

⑥ 빌드 결과

제공 예제는 매뉴얼 코드 그대로이므로 실험 당일 각각 넣어 빌드 · 다운로드한다. 같은 함수를 쓰는 과제3 · 과제4 빌드가 오류 0개로 끝나 예제 코드도 문제없이 컴파일될 것으로 본다.

⑦ 보드 실습 계획

1. 예제 ①: 버튼을 누를 때마다 LD2가 한 번씩 켜짐/꺼짐이 바뀌는지, 빠르게 연타해도 200 ms 안의 입력은 무시되는지 본다.
2. 예제 ②: 버튼 코드는 비운 상태로, LD2가 1초 켜짐 · 1초 꺼짐(주기 2 s)인지 본다.
3. 예제 ③: PB4 · GND에 오실로스코프를 연결해 주기 1.000 ms, HIGH 0.500 ms를 잰다.

5. 실험 5 · STM32_1 과제3 — STM32_0 과제1을 ISR로 재현

① 목적

STM32_0 과제1(버튼을 누를 때마다 LED 반전 간격 1 → 2 → 4 → 8 → 1초)을 메인 루프 없이 **버튼 인터럽트와 TIM6 100 ms 인터럽트**만으로 구현한다. 두 ISR이 값을 함께 쓰는 방법(`volatile` 전역 변수)을 익힌다.

② 이론 · 설계 계산

역할 나누기. 버튼 ISR은 "간격을 바꾼다"만, TIM6 ISR은 "틱을 세다가 간격이 차면 LED를 뒤집는다"만 한다. 간격은 TIM6 틱 단위로 저장한다.

$$T_{\text{반전}} = \text{interval_ticks} \times 100\ \text{ms}, \qquad \text{interval_ticks} \in \{10,\ 20,\ 40,\ 80\}$$

LED의 깜빡임 주기는 켜짐 + 꺼짐이므로 $2T_{\text{반전}}$이다. 버튼 ISR은 `(interval_ticks >= 80) ? 10 : interval_ticks * 2` 로 순환시킨다.

공유 변수와 volatile. `interval_ticks`는 버튼 ISR이 쓰고 TIM6 ISR이 읽으므로 두 함수 밖(파일 범위)에 둔다. `volatile`은 "이 값은 다른 실행 흐름이 언제든 바꿀 수 있으니 매번 메모리에서 다시 읽어라"라는 표시다. 32비트 값 하나를 읽고 쓰는 것은 Cortex-M4에서 명령 하나라 중간에 끊기지 않는다. 다만 volatile이 여러 연산을 한 덩어리로 만들어 주지는 않는다(매뉴얼 주의 사항). `count`는 TIM6 ISR만 쓰므로 함수 안 `static`으로 둔다.

왜 `>=`인가. 8초(80틱) 간격으로 50틱까지 센 상태에서 1초(10틱)로 줄이면, `==`는 count가 10과 같아질 때까지 32비트가 한 바퀴 도는 2^{32}틱(약 13.6년)을 기다린다. `>=`는 다음 틱(51 ≥ 10)에 바로 뒤집고 0부터 다시 센다.

떨림 처리. TIM6(4)이 버튼(7)보다 우선순위가 높아 버튼 ISR 도중에 끼어들 수 있지만, 공유 값은 한 번에 쓰이므로 반쯤 바뀐 값을 읽지 않는다. 버튼 접점 떨림은 마지막으로 인정한 시각과 비교해 200 ms 안의 호출을 버린다.

③ 포트 · 보드 매핑

신호	핀 · 자원	설정
버튼 입력	PC13 (B1 파란 버튼, 누르면 HIGH)	EXTI 상승 에지, 우선순위 7
LED 출력	PA5 (LD2, SET = 켜짐)	푸시풀 출력
100 ms 틱	TIM6	PSC 9999 · ARR 999, 우선순위 4

④ 핵심 코드 (템플릿과 다른 부분)

전체: `LAB5/stm32/task3/ece2_474_routine.c` . `polling_routine()` 은 템플릿 그대로 비어 있다.

```
/* STM32_1_과제3: 두 ISR이 함께 쓰는 LED 간격 (100 ms 틱 단위, 10 = 1초).
 * 버튼 ISR이 바꾸고 TIM6 ISR이 읽으므로 함수 밖에 두고 volatile을 붙인다. */
static volatile uint32_t interval_ticks = 10U;

void ece2_474_init(void)
{
    HAL_TIM_Base_Start_IT(&htim6);   /* STM32_1_과제3 */
}

static void button_isr_p7(void)
{
    static uint32_t last_ms = 0;
    uint32_t now_ms = HAL_GetTick();
    if (now_ms - last_ms >= 200U)          /* 200 ms 안의 반복 호출(접점 떨림)은 무시 */
    {
        last_ms = now_ms;
        /* 1초 -> 2초 -> 4초 -> 8초 -> 1초 */
        interval_ticks = (interval_ticks >= 80U) ? 10U : interval_ticks * 2U;
    }
}

static void tim6_isr_p4(void)
{
    static uint32_t count = 0;             /* 100 ms 틱 수. 이 함수만 사용 */
    if (++count >= interval_ticks)         /* >= : 간격이 갑자기 줄어도 다음 틱에 반전 */
    {
        count = 0;
        HAL_GPIO_TogglePin(LED_ece2_GPIO_Port, LED_ece2_Pin);
    }
}
```

⑤ 시뮬레이션 (호스트 모의 실행)

저장소의 `ece2_474_routine.c`를 **한 글자도 바꾸지 않고** PC의 gcc로 컴파일했다. `main.h`만 PC용 가짜 헤더로 바꿔 `HAL_GetTick` · `HAL_GPIO_TogglePin` · `HAL_TIM_Base_Start_IT`를 흉내 냈다. 가짜 시계는 1 ms씩 가고, 100 ms마다 실제 HAL처럼 `HAL_TIM_PeriodElapsedCallback(&htim6)`을, 버튼 상승 에지마다 `HAL_GPIO_EXTI_Callback(BUTTON_ece2_Pin)`을 부른다. 시나리오는 10.050 s 누름(+3 ms, +7 ms 떨림), 26.050 s, 42.050 s, 75.050 s 누름이다.

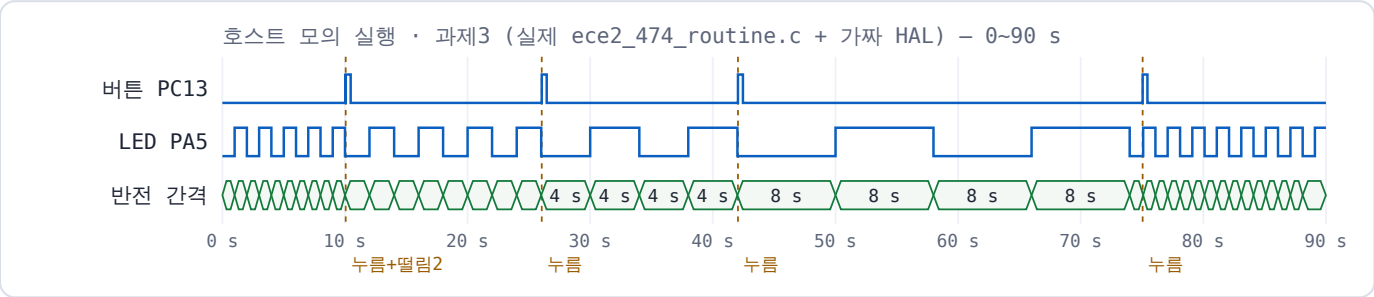


그림 5-1. 과제3 모의 실행. "반전 간격"은 LED 반전 시각 사이의 측정값이다. 떨림 두 번은 무시되어 간격이 한 번만 바뀌었고, 1 → 2 → 4 → 8 → 1 s로 순환했다.

구간	측정된 반전 간격	판정
0 ~ 10 s (시작)	1.0 s × 10	**1 s**
10.05 s 누름 + 떨림 2회 이후	2 s (10 → 12 s), 이후 2.0 s 반복	**2 s, 한 번만 바뀜**
26.05 s 누름 이후	4.0 s 반복	**4 s**
42.05 s 누름 이후	8.0 s 반복	**8 s**
75.05 s 누름 (count = 10틱 상태)	74.0 s → **75.1 s** 반전, 이후 1.0 s	**다음 틱에 즉시 반영 (>=)**

버튼을 누르면 새 간격은 **마지막 반전 시각부터** 센다. 예를 들어 10.05 s에 2초로 바꾸면 10.0 s 반전으로부터 2초 뒤인 12.0 s에 다음 반전이 온다. 누른 시각부터 다시 세려면 버튼 ISR에서 count도 0으로 돌려야 하지만, count는 TIM6 ISR 안의 static이라 그대로 두었다. 과제 요구("선택한 간격마다 LED 상태를 바꾼다")에는 두 방식 모두 맞다.

⑥ 빌드 결과

실제 프로젝트(`build/Debug`, GCC 14.3.1)에서 오류 · 경고 0개로 빌드했다. `arm-none-eabi-size` 결과는 text 25,868 B · data 108 B · bss 2,548 B이다. FLASH 사용 = text + data = 25,976 B(512 KB의 4.95 %), RAM = data + bss = 2,656 B이다.

⑦ 보드 실습 계획

1. PowerShell `run5 3`으로 빌드 · 다운로드 · Serial(Hello, World!) 확인.
2. LD2가 1초 간격으로 깜빡이는지, 파란 버튼을 누를 때마다 2 · 4 · 8 · 1초로 바뀌는지 스톱워치로 잰다.
3. 버튼을 짧게 여러 번 연타해 한 번 누를 때 한 단계만 바뀌는지 본다.

6. 실험 6 · STM32_1 과제4 — 버튼으로 PWM 비율 변경 · C 디버거

① 목적

버튼을 누를 때마다 PB4의 1 kHz PWM 비율을 25 → 50 → 75 → 25 %로 바꾼다. VS Code 디버거의 중단점 · 변수 확인 · 한 줄 실행을 써서, 버튼 ISR이 불리는 순간과 비교값이 바뀌는 과정을 직접 확인한다.

② 이론 · 설계 계산

PWM. TIM3은 1 MHz(100 MHz ÷ 100)로 0부터 999까지 세고 다시 0으로 돌아간다(1 ms). PWM1 모드에서는 카운터가 비교값(CCR1)보다 작은 동안 출력이 HIGH다. 따라서

$$D = \frac{CCR1}{ARR + 1} = \frac{CCR1}{1000}, \qquad CCR1 = \text{pwm_percent} \times 10 \in \{250,\ 500,\ 750\}$$

HIGH 시간은 0.25 / 0.50 / 0.75 ms이다. 문틈을 1 ms마다 얼마 동안 여느냐로 평균 밝기(평균 전압 $D \times 3.3$ V = 0.825 / 1.65 / 2.475 V)를 정하는 것과 같다.

비교값이 바뀌는 시점. HAL의 `HAL_TIM_PWM_ConfigChannel()`은 CCR1 프리로드(`TIM_CCMR1_OC1PE`)를 켠다[2]. 그래서 주기 중간에 비교값을 써도 실제 출력은 **다음 주기 시작(카운터가 0이 될 때)**부터 바뀌고, 한 주기 안에서 파형이 잘리는 일이 없다.

디버거로 볼 변수. "현재 PWM 비율을 저장한 변수"를 Watch에서 바로 읽을 수 있도록 `pwm_percent`(25/50/75)를 파일 범위에 두고, 비교값은 `pwm_percent × 10`으로 계산한다. 디버거가 언제든 메모리에서 읽고 쓸 수 있도록 `volatile`을 붙였다.

③ 포트 · 보드 매핑

신호	핀 · 자원	설정
버튼 입력	PC13 (B1)	EXTI 상승 에지, 우선순위 7
PWM 출력	PB4 = `PWM_OUT_ece2` (TIM3 CH1) → 오실로스코프 CH1, GND는 CN6 GND	PSC 99 · ARR 999 · PWM1 · 프리로드

④ 핵심 코드 (템플릿과 다른 부분)

전체: `LAB5/stm32/task4/ece2_474_routine.c`. TIM6은 쓰지 않는다.

```
/* STM32_1_과제4: 현재 PWM 비율(%). 디버거 Watch에서 확인할 변수.
 * TIM3 한 주기 = 1000칸(ARR 999)이므로 비교값 = 비율 x 10. */
static volatile uint32_t pwm_percent = 25U;

void ece2_474_init(void)
{
    HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1);   /* STM32_1_과제4 */
    __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, pwm_percent * 10U);   /* 시작 25 % */
}

static void button_isr_p7(void)
{
    static uint32_t last_ms = 0;
    uint32_t now_ms = HAL_GetTick();
    if (now_ms - last_ms >= 200U)          /* 200 ms 안의 반복 호출(접점 떨림)은 무시 */
    {
        last_ms = now_ms;
        /* 25 % -> 50 % -> 75 % -> 25 % */
        pwm_percent = (pwm_percent >= 75U) ? 25U : pwm_percent + 25U;
        __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, pwm_percent * 10U);   /* 중단점(F9) 위치 */
    }
}
```

⑤ 시뮬레이션 (호스트 모의 실행)

과제3과 같은 방법으로 저장소 파일을 그대로 컴파일했다. 가짜 `__HAL_TIM_SET_COMPARE`는 htim3의 CCR 값을 기록한다. 시나리오는 1.000 s 누름(+3 ms, +50 ms 떨림), 2 · 3 · 4 s 누름이다. 그 결과로 얻은 비교값 순서를 그림 6-2의 PB4 출력 모델에 넣었다.

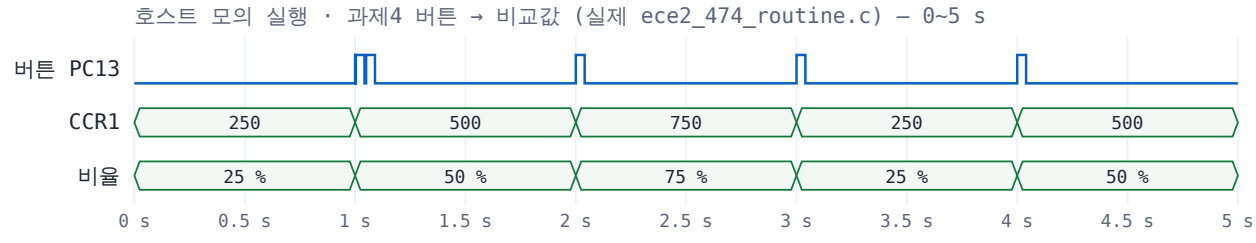


그림 6-1. 과제4 모의 실행. 시작 25 %(CCR1 250)에서 버튼마다 50 → 75 → 25 → 50 %로 순환했다. 1.003 s와 1.050 s의 떨림은 200 ms 안이라 무시되었다.

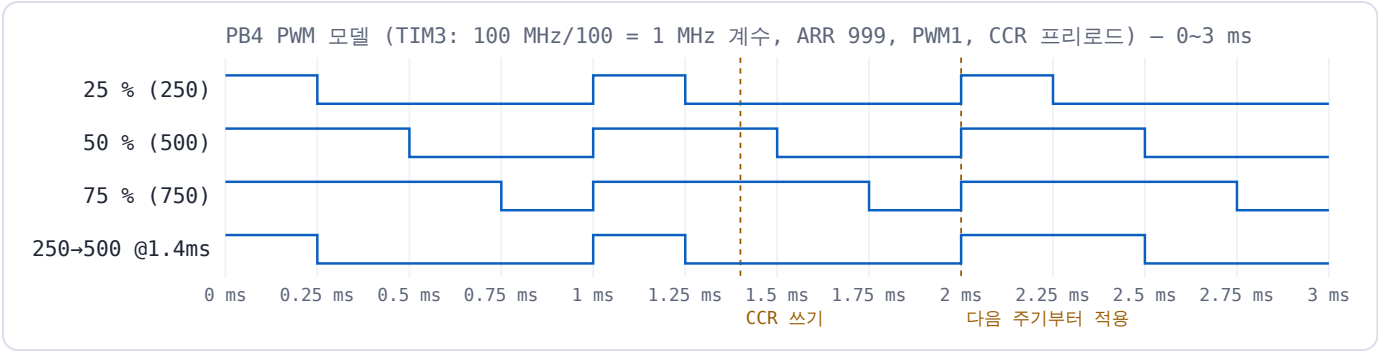


그림 6-2. PB4 출력 모델. 비율별 HIGH 시간은 0.25 / 0.50 / 0.75 ms이다. 맨 아래는 1.4 ms(주기 중간)에 250 → 500을 쓴 경우로, 프리로드 때문에 2 ms의 다음 주기부터 50 %가 된다.

⑥ 빌드 결과

오류 · 경고 0개. text 26,200 B · data 108 B · bss 2,540 B로, FLASH 26,308 B(5.02 %) · RAM 2,648 B이다. 과제3보다 FLASH가 332 B 큰 것은 `HAL_TIM_PWM_Start`가 새로 링크되었기 때문이다.

⑦ 보드 실습 계획 · 디버거 사용 순서

단계	조작	예상 결과
1	`__HAL_TIM_SET_COMPARE(...)`가 있는 버튼 ISR 줄에 **F9**로 중단점	줄 왼쪽에 빨간 점
2	**F5**로 디버깅 시작 → main에서 멈추면 **F5** 한 번 더 → 보드 버튼 누름	중단점 줄에서 멈춤(호출 스택: EXTI15_10_IRQHandler → HAL_GPIO_EXTI_IRQHandler → HAL_GPIO_EXTI_Callback → button_isr_p7)
3	Watch에 `pwm_percent`, `htim3.Instance->CCR1` 추가	바로 윗줄이 실행된 상태라 `pwm_percent = 50`, CCR1은 아직 `250`
4	**F10**으로 한 줄 실행	CCR1 = `500`. 이어서 F5로 계속 실행
5	중단점 해제 후 버튼을 누르며 오실로스코프 확인 → **Shift+F5**로 종료	HIGH 0.25 → 0.50 → 0.75 → 0.25 ms 순환, 주기 1 ms 유지

중단점으로 멈춘 동안에도 TIM3는 기본 설정에서 계속 돌기 때문에 PWM 출력은 멈추지 않는다. 그래도 매뉴얼에 따라 파형 측정은 중단점을 해제한 정상 실행 상태에서 한다. 과제4는 디버거가 필요하므로 이 부분만 VS Code에서 하고, 다운로드 확인은 PowerShell `run5 4`로도 할 수 있다.

고찰

1. **두 시뮬레이터로 같은 TB를 돌린 의미.** Icarus와 XSim의 VCD를 자동 비교해 설계 출력의 모든 변화가 같음을 확인했다(피아노 piezo 34개, ADC sclk 98개 에지 등). 유일한 차이는 시간 0의 조합 출력 X였다. 이 차이가 출력에 남지 않은 것은 수업 TB가 리셋 동안 결과를 검사하지 않고, 입력을 클록 하강 에지에서 바꾸기 때문이다.
2. **나눗셈 한 줄의 비용.** 피아노는 LUT 646개로 신호등(33개) · ADC(60개)보다 훨씬 크다. 원인은 반주기 계산의 변수 나눗셈이다. HDL에서 `/`는 편하지만, 나누는 수가 상수가 아니면 큰 회로가 된다. 값의 종류가 적으면 표로 미리 계산하는 편이 낫다.
3. **ISR은 짧게.** 두 과제 모두 ISR은 값 비교 · 대입 몇 줄만 한다. SysTick이 우선순위 15(가장 낮음)라서 ISR 안에서는 시간이 멈춰 있고, 그래서 HAL_Delay가 금지된다. 떨림 처리도 기다리지 않고 "시각을 비교해 버리는" 방식으로 했다.
4. **volatile의 범위.** volatile은 컴파일러가 값을 레지스터에 묶어 두지 못하게 할 뿐이다. 과제3에서는 32비트 값 하나만 주고받아 충분하지만, 두 값을 함께 바꾸거나 "읽고-고치고-쓰기"를 나눠 하면 중간에 끼어든 ISR 때문에 값이 어긋날 수 있다. 그때는 잠시 인터럽트를 막는 등의 처리가 필요하다.
5. **모의 실행의 한계.** 모의 실행은 함수의 논리와 시간 계산을 확인할 뿐, 실제 버튼의 떨림 시간 · 인터럽트 지연 · 오실로스코프 파형은 보드에서만 확인할 수 있다. 실험 후 보고서에서 측정값과 비교한다.

참고문헌

1. [1] D. M. Harris and S. L. Harris, *Digital Design and Computer Architecture*, 2nd ed. Waltham, MA, USA: Morgan Kaufmann, 2013. ISBN 978-0-12-394424-5. — Ch. 3 순서 회로 · 유한 상태 기계(신호등), Ch. 5 카운터, Ch. 8 I/O의 SPI · PWM · 인터럽트.
2. [2] STMicroelectronics, *stm32g4xx-hal-driver*, GitHub, https://github.com/STMicroelectronics/stm32g4xx-hal-driver, 태그 v1.2.7 (스타 53, 확인일 2026-10-17). — `stm32g4xx_hal_tim.c`의 `HAL_TIM_Base_Start_IT` · `HAL_TIM_PWM_Start` · `HAL_TIM_PWM_ConfigChannel`(OC1PE 프리로드 설정), `stm32g4xx_hal_tim.h`의 `__HAL_TIM_SET_COMPARE`.

수업 자료(인용 출처 수에 넣지 않음): Glaysia/ece2_474 v1.1.0의 docs/STM32_1.md · docs/FPGA_additional.md · docs/index.md, 실습 교재 「Combo II DLD 장비를 이용한 디지털 논리 회로 설계」.